Multi-Account Test Strategy — All Surfaces, Anti-Bluff Captured Evidence (to-be §27)

Revision: 1 **Last modified:** 2026-07-10T11:18:54Z

> **Scope.** This document designs the **test strategy** for the multi-account (multi-tenant) Helix OTA feature: every applicable test type per §11.4.27 (no fakes beyond unit) + §11.4.169 (the closed test-type set) + §11.4.107 (anti-bluff AV/validation), across every surface — server API, data-model/RLS, project CLI, device client, dashboard SPA, ota-manager SPA. It is a **design / planning** document: **no test code is written here** (the multi-account work is operator-approval-gated per 00_INDEX §2). Every design choice is grounded in a cited sibling doc or an external source (`## Sources verified 2026-07-10`); every open choice is a recommendation-with-tradeoffs for the owner doc, never a silent decision (§11.4.6 / §11.4.66).
>
> **Reading order + SSOT (do not contradict).** The **entity + RLS SSOT** is `20_target_multitenancy_data_model.md`; the **all-surface UI SSOT** is `23_ui_ux_all_surfaces.md`; the **CLI** and **device-client** designs are `24_project_side_cli.md` / `25_device_side_update_client.md`; the authoritative as-is facts are `10_existing_auth_and_project_model.md` / `11_existing_upload_and_device_update.md` / `12_existing_ui_surfaces_and_design_system.md`. This doc **cites** those, never re-derives their `file:line` facts. The **permission-model shape** (OQ-2) is `21_authz_rbac_superadmin.md`'s and the **threat enumeration** is `26_security_threat_model.md`'s — **`21_*` and `26_*` are planned but not yet authored at this revision** (00_INDEX §3); where this doc names a threat it draws it from `20_*` §6 + the isolation sketches in `24_*` §6 / `25_*` §6 and defers the canonical enumeration to `26_*`, inventing no conflicting threat.

---

0. Grounding — what this strategy tests, and the two hard test-infra facts

The feature under test, from the grounded audits (cited, not re-derived):

- The account layer is primarily a **data-model** + **scoping** effort, not a from-scratch auth build — auth, RBAC, `super_admin`, both sign-in UIs, and a per-project `ProjectAccess` ACL already exist (00_INDEX §7; `10_*` §1-§2; `12_*` §4). The **central gap** is that there is **no tenant isolation**: no `account` entity, and `Device/Artifact/Release/Deployment/Telemetry/Group` carry no `ProjectID`/account field, so every OTA route is gated only by the GLOBAL RBAC role (`10_*` §2, §4).
- The to-be model (`20_*`): **shared schema** + **a denormalized `account_id` column on every tenant-owned table, defended by PostgreSQL Row-Level Security (RLS)** at the pgx layer (`20_*` §0), with **explicit `accountID` params** on the scoped Repository methods as the compile-time first layer and **RLS as the independent second layer** (`20_*` §3.2 “Recommendation (hybrid)”).

Two test-infrastructure facts that shape every layer below (§11.4.6, stated so they are never silently assumed):

1. **RLS is pgx-only; the default store is in-memory.** `MemoryRepository` is wired by default; `PostgresRepository` only when `HELIX_DATABASE_URL` is set (`10_*` §5; `20_*` §3). RLS (the database-enforced belt) **exists only on the pgx path**. So: the **application-layer scope test** (explicit-`accountID` scoping) MUST run against **BOTH** backends; the **RLS-layer test** is **pgx-only** and requires a **live PostgreSQL**, booted on-demand via the **containers submodule in rootless podman (§11.4.161)** — never a mock (§11.4.27). Absent Postgres, the RLS layer is an honest **`SKIP-with-reason: topology_unsupported`** per §11.4.3, **never a PASS-by-default**.
2. **The “device downloads + applies the exact bytes” e2e is blocked on object storage.** Artifact bytes are validated then **discarded**; `StorageRef` is a placeholder and no real object store is in-repo (`11_*` §Honest-gaps 1; `24_*` §4; `25_*` §5). So the isolation / scope / UI / super-admin / migration layers are all validatable **now**, but the byte-level download-and-apply e2e is honestly **`SKIP-with-reason: object_storage_absent`** until real per-account object storage (MinIO dev via rootless podman) lands — never a fake PASS over discarded bytes.

Both facts are carried into the coverage ledger (§6) so the honest state is visible, not green-over-a-gap.

---

1. Test-type matrix × surface

Six surfaces. For each, the applicable test types and the single **anti-bluff proof** each surface must produce (positive captured evidence per §11.4.5 / §11.4.69, never a metadata-only / absence-of-error PASS).

Surface	Applicable test types	The anti-bluff proof this surface must produce
Server API (`server/`, Gin monolith; httptest suite exists per project CLAUDE.md)	unit · integration-against-real-server · e2e · **security / tenant-isolation** · DDoS (auth/upload flood) · concurrency/race · stress+chaos · benchmarking · memory · Challenges/HelixQA	Captured HTTP request+response transcripts + store-row dumps proving a scoped token reads/writes **only its account's** resources; a cross-account call is denied (§2 API layer)
Data model / RLS (`store.Repository` memory + pgx; `20_*`)	unit (memory) · integration (real store, **both** backends) · **RLS-isolation** (pgx, live Postgres) · concurrency/race (monotonicity + active-deployment keys, `CreateAccountWithOwner` atomicity) · migration/backfill (§5) · memory	Captured SQL/query output proving RLS returns **only** the current-account rows **even with a forged `WHERE account_id`**, and 0 rows on a cross-account UPDATE/DELETE (§2 DB layer)
Project CLI (`helix-ota`, new binary; `24_*`)	unit (arg/flag parse, **token-redaction** §11.4.10, exit-code map) · integration (real server) · e2e (real server + real store + object storage) · security (cross-project/-account key → 403, expired/revoked key) ·	Captured server JSON + store dumps proving the artifact persists under the **correct `(account_id, project_id)`**, and a **cross-project key is rejected 403** (`24_*` §6); the raw token **never** appears in

Surface	Applicable test types	The anti-bluff proof this surface must produce
	stress+chaos (concurrent cross-account uploads, kill-mid-upload) · full-automation §11.4.98 · Challenges/HelixQA	any log line (§11.4.10 redaction)
Device client (`ota-android-agent` + Go `ota-device-emu` reference client; `25_*`)	unit (core Kotlin: `VerifyBeforeApply` ordering, wizard state machine, offline/resume) · integration (real `ota-server`) · e2e (real journey, blocked on object storage — §0) · **tenant-isolation** (device A never offered account B's deployment) · UI host-render §11.4.170 (net-new wizard) · stress+chaos (offline→resume, corrupt→reject, power-loss→A/B rollback)	Captured poll request/response proving a device enrolled to account A gets its **own** account's offer and **`204`**/**deny** for account B's deployment even at matching `(os, model, group)` (`25_*` §6)
Dashboard SPA (`dashboard/`, Vite; hostrender harness exists)	unit (component logic) · **UI host-render §11.4.170** (golden-diff + OCR/vision) · integration (auth/account-switch against real server) · e2e	Device-independent **host-rendered pixels** per screen×state×{light,dark}, dual-validated by golden image-diff **and** the OCR/vision layout oracle (§3) — value/token-equality tests FORBIDDEN as sole proof
OTA Manager SPA (`clients/ota-manager/`, React 19 + shadcn; also served at `/manager`;	unit · **UI host-render §11.4.170** · integration · e2e (account switcher re-scopes projects + invalidates caches)	Same host-rendered dual-validated pixel proof (§3) for the account switcher, post-login picker, and super-admin console; plus a wire proof that switching

Surface	Applicable test types	The anti-bluff proof this surface must produce
hostrender + OCR harness exists)		account **re-scopes** the project list and **invalidates account-scoped caches** (`23_*` §2.3)

§11.4.169 closed-set completeness (every type mapped to this feature, none skipped):

§11.4.169 type	This feature's instance	Load-bearing for isolation?
unit	flag/arg parse, redaction, `VerifyBeforeApply` order, rank-compare, scope-resolution helpers (mocks OK here only, §11.4.27)	supporting
integration	real server + real store, scoped token reads only its account	**yes**
e2e	sign-in → scope → publish → device offer (byte-apply blocked on §0.2)	**yes**
full-automation §11.4.98	every layer self-driving, `-count=3`, self-cleaning	supporting
Challenges / HelixQA §11.4.27	one bank entry per user-visible feature driving the real journey	**yes**
DDoS	auth-login brute-force + upload flood vs the rate-limit posture (per-account)	supporting
security / tenant-isolation	**THE flagship** cross-account matrix (§2)	**yes (primary)**
stress + chaos §11.4.85	N concurrent cross-account uploads (no bleed), kill-mid-upload, offline→resume, corrupt→reject, power-loss→A/B rollback	**yes**

§11.4.169 type	This feature's instance	Load-bearing for isolation?
concurrency / race-deadlock	scoped `LatestRelease` monotonicity + `ActiveDeploymentForTarget` uniqueness under concurrent cross-account writes; `CreateAccountWithOwner` atomicity (no orphan tenant)	**yes**
memory	no leak under sustained scoped-session / RLS-GUC set-reset churn	supporting
benchmarking	scope-resolution overhead (Option A per-call key hash+lookup vs Option B token, `24_*` §1.3); RLS overhead vs no-RLS baseline	supporting

Each PASS cites rock-solid captured physical evidence under `docs/qa/<run-id>/` (§11.4.83) via `ab_pass_with_evidence` (§11.4.69) — never a bare `ab_pass`.

---

2. THE flagship test — cross-account isolation (the primary threat)

The single most important proof of the whole feature: **account A cannot see or modify account B's devices / artifacts / releases / deployments** — the primary threat class `20_*` §6 and `24_*`/`25_*` §6 name. It is proven at **three layers**, each with a paired §1.1 mutation that removes the scoping and makes the test FAIL. Isolation asserted at only one layer is not proven — a scoped session must be denied by BOTH the application scope (layer B) AND the database (layer A, pgx), because either alone can be bypassed by the other's bug (`20_*` §0 defense-in-depth).

Shared anti-bluff design (applies to all three layers), the §11.4.107-style not-a-false-negative guard applied to isolation: an "A cannot see B" assertion is a bluff if the query is simply broken (a broken query

returns empty for *everyone*). Every isolation assertion is therefore **three-part in the same call**: (i) A's own rows are **present and non-empty**, (ii) B's rows are **absent**, (iii) the paired §1.1 mutation **flips (i)/(ii)** — B's rows appear. Only all three together prove isolation, not a coincidentally-empty result.

2.1 DB layer — RLS (pgx, live Postgres via rootless podman §11.4.161)

Setup. Boot a real PostgreSQL (containers submodule, rootless podman). Apply the `20_*` §4 schema with RLS enabled: the app connects as a **non-owner, non-`BYPASSRLS` role** and runs `SET app.current_account = '<account_id>'` per session (`20_*` §4 step 8, §6). Seed account **A** and account **B**, each with a device, artifact, release, and deployment (composite-FK-consistent per `20_*` §2).

Assertions (session with `SET app.current_account = A`):

1. `SELECT * FROM devices` (and artifacts/releases/deployments) → returns **A's rows, non-empty** and **zero B rows**.
2. **Forged-`WHERE` read** — `SELECT * FROM devices WHERE account_id = 'B'` → **0 rows** (RLS filters *before* the `WHERE`; this is the belt that "works even when your application code has bugs" — AWS RLS blog).
3. **Cross-account write** — `UPDATE devices SET … WHERE account_id = 'B'` and `DELETE FROM devices WHERE account_id = 'B'` → **0 rows affected**.
4. **Determinism** — the same suite at `-count=3` yields identical row sets + identical captured-evidence hashes (§11.4.50 / §11.4.98).

§1.1 mutation (proves the gate is not a bluff): drop the RLS policy on `devices` (or weaken it to `USING (true)`, or omit the per-session `SET app.current_account`) → assertion 2's forged-`WHERE` read **returns B's rows** → the test **FAILs**. Restore → GREEN. A policy that still hides B's rows after being weakened would itself be a bluff gate (§11.4.120 reconcile-don't-fake).

Evidence (§11.4.69): captured `psql`/driver query output for each assertion + the seeded row dumps + the mutation-run output showing the flip, under `docs/qa/<run-id>/isolation_rls/`.

2.2 API layer — scoped routes (real `ota-server`, httptest, both store backends)

Setup. Boot the real server with a real store; mint a **scoped access token for a user in account A** (the `20_*`/`21_*` account claim, or Option B token-exchange per `24_*` §1.3). Seed A's and B's resources through the real API.

Assertions:

1. `GET /devices`, `GET /releases`, `GET /deployments`, `GET /artifacts?project=…` (the list-artifacts endpoint `24_*` §4 adds) with A's token → **only A's resources** (non-empty A, zero B).
2. `GET /devices/{B_device_id}` (a **known** id in account B) with A's token → **denied** (recommend **404** to avoid an existence-leak; **403** for an in-account operation the role lacks — the 404-vs-403 choice is `22_*`/`26_*`'s to bind, flagged not decided; `24_*`/`25_*` §6 use "403/deny").
3. `PATCH`/`DELETE /devices/{B_device_id}` with A's token → denied; B's row is **unchanged** (re-read as a super-admin or a B-scoped token to confirm no mutation).
4. **Cross-project / cross-account key on write** — a key scoped to account/project X used on `POST /artifacts/upload` / `POST /releases` / `POST /deployments` for Y → **403** (`24_*` §6). The **account is taken from the resolved token/key scope, never from a request field** (`24_*` §3 step 2 — the trust boundary, same rule as `resolvePublicKey`).

§1.1 mutation: remove `requireAccountAccess` from the route (or drop the `AccountID` filter from the handler's Repository query / `*Filter` struct, `20_*` §3.2) → A's token **sees/mutates B's resources** → the test **FAILs**. Restore → GREEN.

Evidence: captured HTTP request+response JSON transcripts (headers + bodies) for every assertion + store-row dumps before/after the write attempts, under `docs/qa/<run-id>/isolation_api/`. Run against **both** memory and pgx backends (the memory backend has no RLS, so the app-layer scope is its *only* isolation — layer 2.2 is where the memory MVP's isolation is proven).

2.3 E2E layer — the real user/device journey

Setup. Provision two accounts end-to-end. A user in A signs in via the SPA (or the CLI `whoami`), and a device is enrolled to A (operator-minted scoped token, `25_*` §1.3 Option A).

Assertions:

1. **User** in A sees **only A's** projects/devices/releases in the SPA (host-render + OCR of the scoped list, §3) and via `helix-ota list` commands (`--json`, `24_*` §2).
2. **Device** enrolled to A polling `GET /client/update` receives **A's** active deployment and **`204`/deny** for B's deployment **even when `(os, model, group)` match** — the exact cross-tenant-leak-by-construction `25_*` §1.1 warns of, now closed by the account/project-scoped `ActiveDeploymentForTarget` (`25_*` §1.2).
3. The device on the **other** account (B) is **unaffected** by A's release.

§1.1 mutation: remove the account/project filter from `ActiveDeploymentForTarget` (revert to the global `(os, model, group)` key, `11_*` §4) → device A is **offered B's deployment** → the e2e **FAILs**. Restore → GREEN.

Evidence: device-emulator poll transcripts + the offered manifest + (SPA) the host-rendered scoped-list PNG/OCR, under `docs/qa/<run-id>/isolation_e2e/`. The **byte-level** "device downloads + applies A's exact artifact" step is honestly `SKIP-with-reason: object_storage_absent` until §0.2 lands — the **routing/offer isolation** above is fully provable now; the byte-apply is not, and that is stated, not silently green (`24_*` §4 / `25_*` §5).

---

3. UI proof (§11.4.170) — sign-in split, account switcher, super-admin console

Every new/changed UI screen MUST be proven by **device-independent host-rendered pixels** (the real component rendered to a PNG on the host — no device/emulator), **dual-validated** by (i) a **golden image-diff** AND (ii) an **OCR/vision oracle** reading rendered text + labels + control bounds (no overlap / label-over-label / clipping / off-screen / collapsed-or-giant-unbounded widget). **Value/token-equality unit tests are**

FORBIDDEN as the sole proof (§11.4.170 forensic: hex/sp/dp value-equality tests stayed GREEN while a visibly-broken screen shipped); they MAY supplement, never substitute.

3.1 The harnesses ALREADY EXIST — new screens add fixtures to them

This is grounded in `23_*` §4 (re-verified there this session); the test strategy **extends** these, it does not build a new harness:

- **OTA Manager — `clients/ota-manager/visual/`:** `harness.tsx` + `run-all.mjs` render components to PNG on the host (Storybook-class); **`oracle-diff.mjs`** = **the golden image-diff** oracle; **`oracle-ocr.mjs`** = **the OCR/vision layout oracle**; `vite.harness.config.ts`. It also ships a **dashboard variant** (`harness-dashboard.tsx`, `run-all-dashboard.mjs`) so the OCR/vision oracle can cover dashboard screens too.
- **Dashboard — `dashboard/hostrender/`:** Playwright specs (`login.hostrender.spec.ts`, `screens.hostrender.spec.ts`) with `-snapshots/` dirs (Playwright `toHaveScreenshot` = the golden image-diff) + `playwright.hostrender.config.ts`.

3.2 Fixture matrix — per screen × state × {light, dark} (from `23_*` §4.3)

Screen	States to prove	Surface(s)
Regular sign-in (unchanged form, **re-proven** — the split is a post-auth router concern, `23_*` §1.2 Option A)	idle · validation-error · submitting	both SPAs
Account picker (post-login, `23_*` §2.1)	multi-account list · no-membership error · (single-account = auto-select, **no screen**)	both SPAs
AccountSwitcher (above the ProjectSwitcher, `23_*` §2.2)	closed · open · active-account checked · **long-name truncation**	both SPAs
		both SPAs

Screen	States to prove	Surface(s)
Super-admin: accounts list	populated · empty · loading	
Super-admin: account create/edit dialog	empty · filled · error	both SPAs
Super-admin: users list + user create/edit	populated · membership-assignment open	both SPAs
Super-admin: **permissions matrix** (roles × resource×action grid, `23_*` §3.2)	full grid · **horizontal-scroll, no clip/ overlap of header or first data row**	both SPAs
(Option B only) super-admin sign-in	idle · error	only if the operator picks the `23_*` §1.2 Option-B split

Dashboard precondition (flag, from `23_*` §5): the dashboard `Role` union OMITS `super_admin` (`dashboard/src/types/api.ts`); the super-admin console cannot be gated/rendered on the dashboard until `super_admin` is added to that union (and to `TokenResponse.roles`). The dashboard super-admin fixtures are **blocked** on that small type change — a documented ordering dependency, not a test gap to paper over.

3.3 Anti-bluff for the oracles themselves (§11.4.107(10) self-validation)

The OCR/vision oracle (`oracle-ocr.mjs`) and the golden-diff oracle (`oracle-diff.mjs` / Playwright) are themselves proven not-a-bluff: a **golden-good** fixture (a correctly-laid screen) MUST PASS, and a **golden-bad** fixture (an overlapping / clipped / label-over-label screen) MUST FAIL, pinpointing the offending region — wired into meta-test. **§1.1 mutation:** feed the golden-bad fixture to the oracle; if it PASSes, the oracle is the bluff and the gate FAILs (§11.4.107(10) analyzer self-validation). Thresholds (`maxDiffPixelRatio` / OCR per-word confidence floor) are **calibrated on the project's own fixtures, not hardcoded from literature** (§11.4.6 / §11.4.107(13)); rendering is pinned to one OS/browser environment (Playwright's Docker image / the pinned

harness env) so the goldens are deterministic (Playwright docs: rendering varies by host OS/version — run in the same environment the baseline was generated in; §Sources).

Honest boundary (§11.4.170): these are **host-rendered** device-independent pixels — proof the real component *renders* correctly per state and theme. It is **not** the running wire proof that the switch re-scopes data (that is §2.2/§2.3 and the §11.4.169 e2e). Both are required; neither substitutes the other.

---

4. Super-admin tests (bypass only for super-admin · audit written · no escalation)

The super-admin is a **global `users.is_super_admin` boolean** — a property of the identity, not a per-account role — and it bypasses account scoping at **two layers** (`20_*` §6). The tests prove the bypass is **scoped to super-admins, audited, and revocable**, never a standing hole.

1. **Bypass works only for super-admin.** A super-admin session `ListAccounts` / `ListUsers` returns **all** accounts/users (the app-layer resolver returns "all accounts" for super-admin — the exact precedent `handlers_project.go`'s global-admin bypass, `20_*` §6). A **non-super-admin** session calling those admin routes → **403**. *§1.1 mutation:* grant the bypass to a non-super-admin role → the non-super-admin `ListAccounts` returns all accounts → **FAIL**.
2. **RLS bypass is policy-based, not `BYPASSRLS`, and is immediately revocable (`20_*` §6).** A super-admin request sets a super-admin GUC (`SET app.is_super_admin = 'on'`) that each policy's `USING` clause also accepts → sees all accounts' rows. **Clearing the GUC re-scopes the SAME session immediately** to its own account only. *§1.1 mutation:* replace the policy-based bypass with a `BYPASSRLS` role **OR** make the bypass survive the GUC-clear → the revocation assertion **FAILs** (a super-admin bypass that survives revocation is a §11.4 isolation-layer bluff, `20_*` §6). This also proves **auditability**: a `BYPASSRLS` role is invisible to the policy and cannot be attributed per query — the policy-based path keeps every super-admin query attributable (AWS RLS blog: "policy-based admin access … keeps admin access auditable and revocable"; §Sources).
3. **Audit log is written with the affected `account_id`.** Every super-admin mutating action records an audit row carrying the **affected tenant's `account_id`** and the actor (`20_*` §6; the audit table today

carries neither `account_id` nor a populated `UserID`, `10_*` §6). *§1.1 mutation:* drop `account_id` from the audit write → the "super-admin action is tenant-attributable" assertion **FAILs**.

4. **A non-super-admin cannot escalate.** `is_super_admin` is settable **only via config/env bootstrap, never a request** (`20_*` §6; the same trust boundary as the token secret / TLS-proxy flag / `resolvePublicKey`, `10_*` §7). Tests:
   1. an account `admin` calling `PATCH /users/{id}` with `is_super_admin=true` → **denied**;
   2. a **forged `is_super_admin` claim** in a self-minted / tampered token → **ignored** (the claim is server-minted + server-verified, never trusted from the request);
   3. an account admin cannot grant itself membership in another account it does not own. *§1.1 mutation:* honor a request-supplied `is_super_admin` field/claim → the privilege-escalation test **FAILs**.

Evidence: captured HTTP transcripts + audit-row dumps + the RLS session output (GUC-on → all rows, GUC-off → own rows) under `docs/qa/<run-id>/superadmin/`. **Bootstrap-only note (§11.4.10):** the first super-admin is seeded from a `HELIX_*` env field (`20_*` §6; `10_*` §7) — the escalation tests never create one via the API, they prove the API *cannot*.

---

5. Migration / backfill tests — OQ-4 default-account backfill correctness

OQ-4 (00_INDEX §5): migrate existing projects/rollouts/artifacts under a **default account**, or keep the schema additive-nullable until backfill? `20_*` §5 **recommends Option A (default-account big-bang, then NOT NULL)** because the as-is store is in-memory-by-default and barely populated, making the backfill nearly free, and it removes the nullable-isolation-hole window entirely. This section tests **Option A** (and the Option-B variant if the operator picks it), on the **pgx path** (a live Postgres via rootless podman; the memory backend has an analogous in-code default-account assignment).

Setup. Seed a **pre-migration** store with existing rows carrying **no `account_id`**: projects, devices, artifacts, releases, deployments, groups, telemetry, audit (the eight tenant-owned tables of `20_*` §2.1). Run the migration sequence: `002` (add `account_id` nullable) → **backfill** (assign every legacy row to the seeded `__default__` account, and a `__default__`

project where a `project_id` is required) → `003` (set `account_id NOT NULL`, add the per-account uniques, the composite FKs, the leading-column indexes, and enable RLS) — `20_*` §5 Option A.

Assertions (backfill correctness):

1. **No orphan / no NULL after `003`.** Every legacy row now has `account_id = __default__`; the `003` `NOT NULL` add **succeeds** (it would FAIL if any row were left NULL — that failure IS a detection). Assert `COUNT(* WHERE account_id IS NULL) = 0` on every migrated table.
2. **No cross-account bleed.** With a single default account, every migrated row lands under `__default__` and **none** under any other account: `COUNT(rows under __default__) == COUNT(pre-migration rows)` per table, and `COUNT(rows outside __default__) == 0`.
3. **Composite-FK integrity holds** (`20_*` §2): every resource's `account_id` **equals** its project's `account_id` (the `(project_id, account_id)` composite FK the `003` migration adds) — assert 0 violating rows.
4. **Uniqueness re-scoping is correct** (`20_*` §2.2): the old global uniques (project name, device `hardware_id`, group name) are dropped and re-added **per-account**; a legacy row set that had one global "production" project now validates under the per-`(account_id, name)` unique.
5. **Round-trip / scoped read.** Post-migration, an app-layer scoped read for `__default__` returns **exactly** the legacy rows; an RLS session with `SET app.current_account = __default__` sees them and a *different* account's session sees **zero** (composes with §2.1).
6. **Idempotent + crash-safe** (§11.4.98 / §11.4.85): running the backfill **twice** yields the identical result; a **SIGKILL mid-backfill** leaves a resumable state (no partial NOT-NULL failure, no half-assigned rows) — captured recovery trace.

§1.1 mutations (each makes a specific assertion FAIL): - **Skip a table in the backfill** (e.g. forget `telemetry_events`) → its `003` NOT-NULL add FAILs OR assertion 1 finds a NULL row → **FAIL**. - **Assign a row to a second account** (bleed) → assertion 2's "0 rows outside `__default__`" **FAILs**. - **Break the composite FK** (set a resource's `account_id` ≠ its project's) → assertion 3 **FAILs**.

Option-B variant (only if the operator chooses additive-nullable-until-backfill, `20_*` §5). Additionally test the interim **NULL-scoping guard**: while `account_id` is NULL, those rows are "unassigned/legacy-global" and MUST be **visible only to super-admin** — a non-super-admin scoped session must see **zero** NULL-account rows. *§1.1*

mutation: make NULL-account rows visible to a non-super-admin scoped session → **FAIL** (this is the isolation-hole `20_*` §5 warns Option B carries during its window).

Evidence: pre/post row-count dumps, the `COUNT(NULL)=0` and `COUNT(outside)=0` query outputs, the composite-FK verification query output, and the twice-run / crash-recovery traces, under `docs/qa/<run-id>/migration/`.

---

6. Anti-bluff discipline for the whole feature

The bar (§11.4 / §11.4.87 / §11.4.126): shipping is **not** "tests pass" but "users can use the feature" — every PASS carries positive captured evidence that the feature works, and every gate is provably not a bluff.

1. **Captured evidence for every PASS (§11.4.69).** Every PASS calls `ab_pass_with_evidence <desc> <path>` citing a real, non-empty artefact under `docs/qa/<run-id>/` (§11.4.83); a bare `ab_pass` is deprecated → FAIL. Evidence shapes per surface: HTTP request+response transcripts (API/CLI/device), store-row + SQL query dumps (data/RLS/migration), host-rendered PNGs + OCR JSON (UI), device-emulator manifests + downloaded-artifact hashes (e2e, when object storage lands), `latency.json` + `categorised_errors.txt` + `recovery_trace.log` (stress/chaos). Metadata-only / config-only / absence-of-error / grep-without-runtime PASS are all forbidden (§11.4 / §11.4.1).
2. **Every gate has a paired §1.1 mutation.** Each isolation / super-admin / migration / UI gate above ships its **mutation that removes the scoping/guard and makes the gate FAIL** (§2.1–§2.3, §4, §5, §3.3). A gate whose mutation does **not** make it FAIL is a bluff gate and is itself a finding; a fix that breaks its own gate is **reconciled, never fake-passed** (§11.4.120).
3. **Deterministic re-runnability (§11.4.98 / §11.4.50).** Every layer below unit is **fully self-driving end-to-end** — the only human step is a one-time credential bootstrap **outside** test execution (env/`.env` per §11.4.10); no "operator types a message" mid-run. Every suite passes at **`-count=3`** with self-cleaning state and identical exit + identical evidence hashes. Live tests drive the account/device programmatically (no hard-coded session UUID colliding with a dev session, per §11.4.98's forensic).
4. **No fakes beyond unit (§11.4.27).** Integration / e2e / security / stress / chaos / UI-host-render / migration all exercise the **real** server + **real** store — the **real PostgreSQL** (rootless podman via

the containers submodule, §11.4.161) for the RLS/pgx/migration paths, the **real `ota-server` binary/httptest**, the **real Go device emulator** for the device journey. Mocks live only in unit tests. Production code never imports a mock path.

5. **Coverage ledger (§11.4.25 / §11.4.52).** One row per **feature × surface × invariant 1–6** (anti-bluff posture · working end-to-end on the real topology · matches the documented promise · no open bug · full docs in sync · four-layer floor), each row classified `AUTONOMOUS_VERIFIED` / `AUTONOMOUS_DESIGNED` / `OPERATOR_ATTENDED_ONLY` / `NOT_APPLICABLE` (§11.4.52). `OPERATOR_ATTENDED_ONLY` is a release blocker until promoted, citing a tracked migration item. Two rows are **honestly non-green today by construction** (§0): the **RLS layer** is `topology_unsupported`-SKIP when Postgres is absent, and the **byte-level device-apply e2e** is `object_storage_absent`-SKIP until real object storage lands — both stated in the ledger, never faked green.
6. **Independent verification + review of the test code itself (§11.4.142 / §11.4.165 / §11.4.134).** The tests are code; a bluff-capable test (one that PASSes on broken-for-the-user behaviour) is a finding. An **independent reviewer/verifier** (structurally separate from the author) confirms every isolation gate genuinely exercises the boundary and its mutation genuinely FAILs, iterating to a zero-finding GO.
7. **Manual QA final confirmation (§11.4.185).** Every automated gate above is necessary but **not sufficient** — the feature is "done" only after the **QA team's manual testing** confirms it, as the final step, after the autonomous suites are GREEN. The agent hands off and waits; it never self-certifies the manual step.

Sources verified 2026-07-10

External best-practice research per §11.4.8 / §11.4.99 — testing multi-tenant isolation (RLS negative-test patterns), tenant-isolation testing strategy (inject-and-assert-denied), and visual-regression testing for auth/switcher flows. Cross-referenced against latest online sources, not copied; each claim above that leans on an external pattern points here.

- **AWS — "Multi-tenant data isolation with PostgreSQL Row Level Security."** https://aws.amazon.com/blogs/database/multi-tenant-data-isolation-with-postgresql-row-level-security/ — verified (also cited by `20_*`): RLS enforces tenant scoping on every SELECT/UPDATE/DELETE/INSERT so it "works even when your application code has bugs" (grounds §2.1's forged-`WHERE` belt), the app role must

not be the table owner or a `BYPASSRLS`/superuser role, and "policy-based admin access over privilege-based bypass … keeps admin access auditable and revocable" (grounds §4's policy-based-bypass + revocability tests).

- **AWS Well-Architected SaaS Lens — REL_3, "How are you testing the multi-tenant capabilities of your SaaS application?"** https://wa.aws.amazon.com/saas.question.REL_3.en.html — verified directly this session: "Create tests that attempt to change the tenant context by injecting a new tenant identifier. Verify that the injection is blocked from crossing a tenant boundary," "inject tenant tokens that attempt to simulate a SaaS identity," and exercise the shared isolation frameworks to "validate that they accurately apply tenant isolation policies." Grounds §2.2's cross-account-token matrix and §4's forged-`is_super_admin`-claim escalation test — a boundary you have not tried to break is a boundary you do not know works.
- **AWS SaaS Architecture Fundamentals — Tenant isolation.** https://docs.aws.amazon.com/whitepapers/latest/saas-architecture-fundamentals/tenant-isolation.html — verified: SaaS systems include explicit constructs that block any attempt to access another tenant's resources even on shared infrastructure, and isolation must be an automated, first-class validation ("attempt cross-tenant access with one tenant's session against another tenant's resources and assert access is denied"). Grounds the three-layer flagship (§2) and the "one layer is not enough" defense-in-depth stance.
- **Playwright — "Visual comparisons" (`toHaveScreenshot`).** https://playwright.dev/docs/test-snapshots — verified directly: golden snapshots are generated on first run (`{test}-{browser}-{platform}.png`) and updated with `--update-snapshots`; `maxDiffPixels`/`maxDiffPixelRatio` set the tolerance (pixelmatch); **"Browser rendering can vary based on the host OS, version, settings, hardware … For consistent screenshots, run tests in the same environment where the baseline screenshots were generated."** Grounds §3.1 (the dashboard `hostrender` golden-diff = `toHaveScreenshot`) and §3.3 (pinned-environment goldens + calibrated-not-hardcoded thresholds).

Corroborating multi-tenant RLS negative-test guidance (add automated isolation tests so a later migration can't silently break tenant scoping; test functions/views/nested queries for unintended policy effects; a consistent per-table tenant-column shape keeps policies testable) surfaced consistently across the RLS-for-multi-tenant corpus (https://www.techbuddies.io/2026/01/01/how-to-implement-postgresql-row-

level-security-for-multi-tenant-saas/, https://dev.to/uaslimcreate/building-multi-tenant-row-level-security-in-postgresql-a-production-pattern-4n2k) and is applied in §2.1 + §5.

No external source prescribes the three-layer (RLS + app-scope + e2e) flagship composition, the not-a-false-negative three-part isolation assertion (§2 shared design), or the default-account backfill correctness matrix (§5) — those are **original work** applied to this codebase's shape, following from `20_*`'s data model.

Honest boundary (§11.4.6)

- **This is a test-strategy design, not implemented.** No test file, harness, fixture, gate, mutation, or evidence artefact described here exists yet; every "assertion" and "§1.1 mutation" is a **plan** for the implementation phase (operator-gated, 00_INDEX §2). Nothing here is a §11.4 completion claim, and no test was run under this doc.
- **The feature-under-test does not exist yet.** Accounts, the `account_id`/`project_id` scoping columns, RLS policies, `requireAccountAccess`, the tenant token claim, the account switcher, the super-admin console, the `helix-ota` CLI, and the device wizard are all **to-be** proposals in `20_*`/`21_*`/`22_*`/`23_*`/`24_*`/`25_*` — this strategy tests them **once they are built**, and reconciles to those docs' final shapes (they are the SSOTs; this doc consumes, never redefines).
- **Two layers are honestly non-green by construction, today (§0):** the **RLS layer** requires a live PostgreSQL and is `topology_unsupported`-SKIP without one (memory MVP has only the app-layer scope, §2.2); the **byte-level device download-and-apply e2e** is `object_storage_absent`-SKIP until real per-account object storage lands (`24_*` §4 / `25_*` §5). The isolation / scope / UI / super-admin / migration proofs are **fully designable and runnable now** without those two.
- **`26_security_threat_model.md` (threat SSOT) and `21_authz_rbac_superadmin.md` (permission-model / token-shape SSOT) are planned but not yet authored** (00_INDEX §3). The threats tested here are drawn from `20_*` §6 + `24_*`/`25_*` §6 and the permission shape from `20_*`'s minimal role skeleton; when `26_*`/`21_*` land, this strategy's threat list and role assertions reconcile to them (they win). Specifically deferred to those docs: the **404-vs-403** cross-account-response choice (§2.2), the exact **token claim names** (§2.2/§4), and the **permission-model shape** the matrix UI renders (§3).

- **The `23_*` "harnesses already exist" claim is inherited, not re-verified here.** §3.1 restates the harness paths (`clients/ota-manager/visual/` with `oracle-diff.mjs` + `oracle-ocr.mjs`; `dashboard/hostrender/` Playwright specs) exactly as `23_*` §4 established them; this doc did not itself re-open those files — it grounds the claim in `23_*`'s verification, and the per-screen tenancy-fixture PASS is the §11.4.170 proof to be produced during implementation, out of scope for a planning doc.
- **The recommendation-vs-decision boundary (§11.4.66) is preserved:** every alternative the owner docs left open (sign-in split A/B, CLI key direct-vs-exchange, device enrollment A/B, OQ-4 A/B, 404-vs-403) is tested along the **recommended** path with the **alternative's** variant noted — this strategy does not pick for the owner docs.